

Comparative Analysis of Naïve Bayes, KNN, and Decision Tree Algorithms in Disaster Prediction

**Joanah Kate Mantiquilla
Al-hamer Palangalan
Armie Joy Tangalin**

BACHELOR OF SCIENCE IN COMPUTER SCIENCE



May 2026

Table of Contents

Problem Definition	3
Mathematical and Computational Model	3
•Dataset Features	4
•Naïve Bayes	5
•K-Nearest Neighbors (KNN)	8
•Decision Tree	12
Decision Tree with 1000 datasets:	16
Implementation and Experimentation	16
1. Dataset Preparation	17
2. Preprocessing	17
3. Model Training	18
4. Prediction	19
5. Evaluation	19
6. Visualization	20
Visualization and Analysis	20
Preprocessing Comparison	21
Accuracy Comparison	22
Confusion Matrix	23
Class Distribution Chart	24
Feature Importance (Decision Tree)	25
Accuracy Trend vs K (for KNN)	26
Evaluation Metrics: Precision, Recall, F1-score	27
Appendices	28
Source Code:	28
Feature Mapping	33
References	34

Problem Definition

Natural disasters such as floods, typhoons, landslides, and earthquakes continue to affect many communities, causing damage to property, disruption of livelihoods, and loss of lives. Accurate disaster prediction and risk classification are important in helping communities and authorities prepare for possible disaster events. With the increasing availability of disaster-related data, machine learning algorithms have become useful tools for analyzing patterns and predicting potential risks.

However, different machine learning algorithms produce varying levels of accuracy, efficiency, and reliability depending on the type and quality of data used. Algorithms such as Naïve Bayes, K-Nearest Neighbors (KNN), and Decision Tree are commonly used for classification and prediction tasks, yet there is limited comparative analysis on their performance in disaster prediction data within academic studies.

This study aims to analyze and compare the performance of Naïve Bayes, KNN, and Decision Tree algorithms using disaster-related datasets. Specifically, the study seeks to determine which algorithm provides better accuracy, prediction performance, and classification capability in disaster prediction. The findings of this study may contribute to future research involving machine learning applications in disaster risk analysis and prediction.

Mathematical and Computational Model

Before getting into the mathematical models, we first took the categorical survey replies and turned them into numerical values, so they would actually match up with the machine learning algorithms. In other words, this “encoding” step lets the qualitative, disaster related variables such as rainfall intensity, soil condition, and evacuation experience, for instance, to show up as quantitative features that the models could handle. Once people’s answers were reshaped into organized numbers, the dataset was basically ready for computational analysis and for predictive modeling, rather than sitting in an unstructured form.

So, this encoding stage works like the bridge between the questionnaire tool and the mathematical models. It shows how raw, human-provided qualitative information from respondents ends up being translated into standardized numerical inputs, and those inputs are then used inside algorithms like Naïve Bayes, KNN, and a Decision Tree during disaster prediction simulations.

- **Dataset Features**

Naïve Bayes

Cloud Density → Low=1, Medium=2, High=3

Humidity → Low=1, Medium=2, High=3

Rainfall (Typhoon) → Light=1, Moderate=2, Heavy=3

Wind Speed → Weak=1, Moderate=2, Strong=3

Typhoon Warning → No=0, Yes=1

KNN

Rainfall Intensity → Very Light=1, Light=2, Moderate=3, Heavy=4, Very Heavy=5

Water Level → Below ankle=1, Knee=2, Waist=3, Chest=4, Above chest=5

Wind Strength → Weak=1, Moderate=2, Strong=3, Very Strong=4

Flood Risk → Low=1, Medium=2, High=3

Decision Tree

Rainfall (Disaster) → Light=1, Moderate=2, Heavy=3

River Overflow → No=0, Yes=1

Soil Condition → Stable=2, Weak=1

Wind Intensity → Weak=1, Moderate=2, Strong=3

Disaster Severity → Low=1, Moderate=2, Severe=3

Additional Survey Variables

Evacuation Experience → No=0, Yes=1

Early Warning System → Always=4, Sometimes=3, Rarely=2, Never=1

Disaster Label → Flood, Typhoon, Landslide, Earthquake, Storm Surge, Fire, Drought

Tech & ML Belief → 1=Strongly Disagree ... 5=Strongly Agree

This section presents the mathematical foundations and computational approaches used by Naïve Bayes, K-Nearest Neighbors (KNN), and Decision Tree algorithms in simulating disaster prediction based on selected environmental features.

- **Naïve Bayes**

Features Used:

Cloud Density, Humidity, Rainfall, Wind Speed → Typhoon Warning

Formula:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

or

$$P(Class|Features) = \frac{P(Features|Class) * P(Class)}{P(Features)}$$

Naïve Bayes is a probabilistic classifier derived from Bayes' Theorem. It assumes that features (like rainfall, humidity, wind speed) are independent. The algorithm calculates the probability of each disaster class (e.g., "Typhoon Warning = Yes") given the observed features, then selects the class with the highest probability.

Assumption:

Features are independent of each other.

Example:

Naïve Bayes				
Cloud Density	Humidity	Rainfall (Typhoon)	Wind Speed	Typhoon Warning
2 (Medium)	2 (Medium)	3 (Heavy)	3 (Strong)	1 (Yes)
3 (High)	3 (High)	3 (Heavy)	3 (Strong)	1 (Yes)
3 (High)	2 (Medium)	3 (Heavy)	3 (Strong)	1 (Yes)
2 (Medium)	2 (Medium)	2 (Moderate)	2 (Moderate)	0 (No)
1 (Low)	1 (Low)	2 (Moderate)	2 (Moderate)	1 (Yes)
2 (Medium)	2 (Medium)	2 (Moderate)	3 (Strong)	1 (Yes)
3 (High)	3 (High)	3 (Heavy)	3 (Strong)	1 (Yes)
3 (High)	3 (High)	3 (Heavy)	3 (Strong)	1 (Yes)
2 (Medium)	2 (Medium)	3 (Heavy)	2 (Moderate)	1 (Yes)
2 (Medium)	2 (Medium)	2 (Moderate)	2 (Moderate)	0 (No)

Test Instance:

Cloud Density = 2 (Medium)

Humidity = 3 (High)

Rainfall = 3 (Heavy)

Wind Speed = 3 (Strong)

Step 1: Compute prior probabilities $P(Class)$

From the dataset:

Yes = 8, No = 2, Total = 10

$$P(Yes) = \frac{8}{10} = 0.8$$

$$P(No) = \frac{2}{10} = 0.2$$

Step 2: Compute likelihoods $P(Feature|Class)$

Cloud Density					Humidity				
	Yes	No	P(Yes)	P(No)		Yes	No	P(Yes)	P(No)
Low	1	0	1/8 = 0.125	0/2 = 0	Low	1	0	1/8 = 0.125	0/2 = 0
Medium	3	2	3/8 = 0.375	2/2 = 1	Medium	4	2	4/8 = 0.5	2/2 = 1
High	4	0	4/8 = 0.5	0/2 = 0	High	3	0	3/8 = 0.375	0/2 = 0

Rainfall (Typhoon)					Wind Speed				
	Yes	No	P(Yes)	P(No)		Yes	No	P(Yes)	P(No)
Moderate	2	2	2/8 = 0.25	2/2 = 1	Moderate	2	2	2/8 = 0.25	2/2 = 1
Heavy	6	0	6/8 = 0.75	0/2 = 0	Strong	6	0	6/8 = 0.75	0/2 = 0

Step 3: Multiply them together

Sample 1 (Medium, Medium, Heavy, Strong → Yes):

$$P(Yes) = 0.375 * 0.5 * 0.75 * 0.75 = 0.11$$

$$P(No) = 1 * 1 * 0 * 0 = 0$$

Sample 2 (High, High, Heavy, Strong → Yes):

$$P(Yes) = 0.5 * 0.375 * 0.75 * 0.75 = 0.11$$

$$P(No) = 0 * 0 * 0 * 0 = 0$$

Sample 3 (High, Medium, Heavy, Strong → Yes):

$$P(Yes) = 0.5 * 0.5 * 0.75 * 0.75 = 0.14$$

$$P(No) = 0 * 1 * 0 * 0 = 0$$

Sample 4 (Medium, Medium, Moderate, Moderate → No):

$$P(Yes) = 0.375 * 0.5 * 0.25 * 0.25 = 0.01$$

$$P(No) = 1 * 1 * 1 * 1 = 1$$

Step 4: Normalize Results

Test Instance (Medium, High, Heavy, Strong)

$$P(Yes) = 0.375 * 0.375 * 0.75 * 0.75 = 0.08$$

$$P(No) = 1 * 0 * 0 * 0 = 0$$

Since:

$$0.08 > 0$$

the predicted output is:

Typhoon Warning = Yes

Final Prediction

For the test instance of (Cloud Density=Medium, Humidity=High, Rainfall (Typhoon)=Heavy, Wind Speed=Strong), the probability of “Yes” is 0.08 while the probability of “No” is 0. Since the highest probability is “Yes”, the model predicts that a typhoon warning should be issued.

- **K-Nearest Neighbors (KNN)**

Before applying the KNN algorithm, the dataset underwent several preprocessing steps to ensure reliability and compatibility with distance-based computation:

Missing values handling

- Survey responses were checked for missing or invalid entries. Rows with incomplete values were either imputed using the median of the feature or removed if deemed inconsistent.

Feature scaling (Min-Max normalization)

- Since KNN relies on Euclidean distance, all features (Rainfall Intensity, Water Level, Wind Strength) were normalized to a common scale between 0 and 1. This prevented features with larger ranges from dominating the distance calculation.

Train-test split

- The dataset was divided into 80% training data and 20% testing data. The training set was used to build the model, while the testing set was reserved for evaluation.

Features Used:

Rainfall Intensity, Water Level, Wind Strength → Flood Risk

Formula (Euclidean Distance):

General n-dimensional form (used in KNN):

$$d(x, y) = \sum (x_i - y_i)^2$$

3-Dimensional Case:

$$d(x, y, z) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

KNN is a distance-based algorithm. It compares the new input to past records by calculating Euclidean distance. The closest k neighbors “vote” for the class. This makes KNN effective for pattern recognition in disaster prediction.

Parameter:

The value of k (e.g., k = 3, k = 5 or k = 7) determines how many neighbors are considered.

Example:

Training Samples (from datasets)

K-Nearest Neighbor				
Sample #	Rainfall Intensity	Water Level	Wind Strength	Flood Risk
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	3 (High)
2	4 (Heavy)	2 (Knee)	3 (Strong)	3 (High)
3	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1 (Low)
4	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1 (Low)
5	4 (Heavy)	1 (Below ankle)	2 (Moderate)	2 (Medium)
6	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1 (Low)
7	4 (Heavy)	3 (Waist)	3 (Strong)	2 (Medium)
8	4 (Heavy)	2 (Knee)	3 (Strong)	1 (Low)
9	4 (Heavy)	1 (Below ankle)	3 (Strong)	2 (Medium)
10	4 (Heavy)	1 (Below ankle)	3 (Strong)	1 (Low)

Test Instance:

Rainfall Intensity = 4 (Heavy) (x)

Water Level = 2 (Knee) (y)

Wind Strength = 3 (Strong) (z)

Step 1: Compute a few distances manually using the 3-Dimensional Case

Sample 1 (4,1,3 → High):

$$d = \sqrt{(4 - 4)^2 + (2 - 1)^2 + (3 - 3)^2} = \sqrt{1} = 1$$

Sample 2 (4,2,3 → High):

$$d = \sqrt{(4 - 4)^2 + (2 - 2)^2 + (3 - 3)^2} = \sqrt{0} = 0$$

Sample 3 (4,1,2 → Low):

$$d = \sqrt{(4 - 4)^2 + (2 - 1)^2 + (3 - 2)^2} = \sqrt{2} \approx 1.41$$

Step 2: Distance Summary Table

K-Nearest Neighbor					
Sample #	Rainfall Intensity	Water Level	Wind Strength	Distance	Flood Risk
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	3 (High)
2	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	3 (High)
3	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1.41	1 (Low)
4	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1.73	1 (Low)
5	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1.41	2 (Medium)
6	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1.73	1 (Low)
7	4 (Heavy)	3 (Waist)	3 (Strong)	1.00	2 (Medium)
8	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	1 (Low)
9	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	2 (Medium)
10	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	1 (Low)

Ascending Order:

K-Nearest Neighbor					
Sample #	Rainfall Intensity	Water Level	Wind Strength	Distance	Flood Risk
2	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	3 (High)
8	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	1 (Low)
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	3 (High)
7	4 (Heavy)	3 (Waist)	3 (Strong)	1.00	2 (Medium)
9	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	2 (Medium)
10	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	1 (Low)
3	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1.41	1 (Low)
5	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1.41	2 (Medium)
4	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1.73	1 (Low)
6	3 (Moderate)	1 (Below ankle)	2 (Moderate)	1.73	1 (Low)

Step 3: KNN Voting

k = 3

K-Nearest Neighbor					
Sample #	Rainfall Intensity	Water Level	Wind Strength	Distance	Flood Risk
2	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	3 (High)
8	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	1 (Low)
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	3 (High)

Nearest are Sample 2 (High), Sample 8 (Low), Sample 1 (High).

→ Majority = **High Flood Risk**.

$k = 5$

K-Nearest Neighbor					
Sample #	Rainfall Intensity	Water Level	Wind Strength	Distance	Flood Risk
2	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	3 (High)
8	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	1 (Low)
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	3 (High)
7	4 (Heavy)	3 (Waist)	3 (Strong)	1.00	2 (Medium)
9	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	2 (Medium)

For $k=5$ → Nearest are Samples 2 (High), 8 (Low), 1 (High), 7 (Medium), 9 (Medium).

→ Votes = High (2), Medium (2), Low (1).

→ Tie resolved by smallest average distance → **High Flood Risk**.

$k = 7$

K-Nearest Neighbor					
Sample #	Rainfall Intensity	Water Level	Wind Strength	Distance	Flood Risk
2	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	3 (High)
8	4 (Heavy)	2 (Knee)	3 (Strong)	0.00	1 (Low)
1	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	3 (High)
7	4 (Heavy)	3 (Waist)	3 (Strong)	1.00	2 (Medium)
9	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	2 (Medium)
10	4 (Heavy)	1 (Below ankle)	3 (Strong)	1.00	1 (Low)
3	4 (Heavy)	1 (Below ankle)	2 (Moderate)	1.41	1 (Low)

For $k = 7$, the model included Samples 2 (High), 8 (Low), 1 (High), 7 (Medium), 9 (Medium), 10 (Low), and 3 (Low), where the majority vote leaned toward **Low Flood Risk** because three of the seven nearest neighbors were labeled Low.

Answer:

For the test instance (Rainfall Intensity=Heavy, Water Level=Knee, Wind Strength=Strong), the KNN model predicts: **Flood Risk = High**

Accuracy comparison across k values according to the test instance:

After testing k=3, k=5, and k=7, the model achieved the following accuracies on the test set:

- k=3 → 100%
- k=5 → 0%
- k=7 → 0%

The best performance showed up at k=3, because most of the near neighbors correctly sided with High flood risk, (2 High vs 1 Low). Smaller k values tend to work better in small datasets, since they lean on the closest points only, and here that setup still gives the right prediction. That said, the larger k settings, like 5 and 7, dragged in more far away neighbors with other labels (Medium and Low). This blurred the influence of the nearest neighbors, and it ended up causing wrong predictions. Therefore, k=3 was picked as the optimal parameter for flood risk prediction in this particular test case.

- **Decision Tree**

Features Used:

Rainfall, River Overflow, Soil Condition, Wind Intensity → Disaster Severity

Splitting Criteria:

Information Gain:

$$Gain(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

Decision Tree is a rule-based model. It splits data into branches using Information Gain. Each branch represents a decision rule (e.g., “If Rainfall > 150 mm AND River Overflow = Yes → High Severity”). This makes the model highly interpretable.

Example:

Decision Tree				
Rainfall (Disaster)	River Overflow	Soil Condition	Wind Intensity	Disaster Severity
3 (Heavy)	1 (Yes)	1 (Weak)	3 (Strong)	3 (Severe)
3 (Heavy)	0 (No)	1 (Weak)	2 (Moderate)	2 (Moderate)
1 (Light)	0 (No)	1 (Weak)	2 (Moderate)	1 (Low)
2 (Moderate)	0 (No)	2 (Stable)	2 (Moderate)	1 (Low)
1 (Light)	0 (No)	1 (Weak)	2 (Moderate)	1 (Low)
1 (Light)	0 (No)	2 (Stable)	2 (Moderate)	1 (Low)
2 (Moderate)	1 (Yes)	2 (Stable)	3 (Strong)	2 (Moderate)
3 (Heavy)	1 (Yes)	2 (Stable)	3 (Strong)	2 (Moderate)
1 (Light)	0 (No)	1 (Weak)	2 (Moderate)	1 (Low)
2 (Moderate)	1 (Yes)	2 (Stable)	2 (Moderate)	1 (Low)

Test Instance:

Rainfall = 3 (Heavy)

River Overflow = 1 (Yes)

Soil Condition = 1 (Weak)

Wind Intensity = 3 (Strong)

Step 1: Calculate entropy of dataset

Compute probabilities of Disaster Severity

$$P(Low) = 6/10 = 0.6$$

$$P(Moderate) = 3/10 = 0.3$$

$$P(Severe) = 1/10 = 0.1$$

Formula:

$$Entropy(S) = -\frac{6}{10} \log_2 \frac{6}{10} - \frac{3}{10} \log_2 \frac{3}{10} - \frac{1}{10} \log_2 \frac{1}{10}$$

$$Entropy(S) = 1.296$$

Step 2: Compute Information Gain for each feature

		Disaster Severity		
		Low	Moderate	Severe
Rainfall (Disaster)	Light	4	0	0
	Moderate	2	1	0
	Heavy	0	2	1
Gain = 0.745				

		Disaster Severity		
		Low	Moderate	Severe
River Overflow	Yes	1	2	1
	No	5	1	0
Gain = 0.306				

		Disaster Severity		
		Low	Moderate	Severe
Soil Condition	Weak	3	1	1
	Stable	3	2	0
Gain = 0.126				

		Disaster Severity		
		Low	Moderate	Severe
Wind Intensity	Moderate	6	1	0
	Strong	0	2	1
Gain = 0.606				

Information Gain formula:

$$H(S) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

After computing the entropy for each split:

- Rainfall → IG = 0.745
- River Overflow → IG = 0.306
- Soil Condition → IG = 0.126
- Wind Intensity → IG = 0.606

Step 3: Split on the feature with the highest gain

		Disaster Severity		
		Low	Moderate	Severe
Rainfall (Disaster)	Light	4	0	0
	Moderate	2	1	0
	Heavy	0	2	1
Gain = 0.745				

Rainfall (Disaster) has the highest gain of 0.745. So, the root node of the decision tree is Rainfall.

Step 4: Follow decision path

If Rainfall = Low → Low Severity

If Rainfall = Moderate → Moderate Severity

If Rainfall = Heavy → check River Overflow

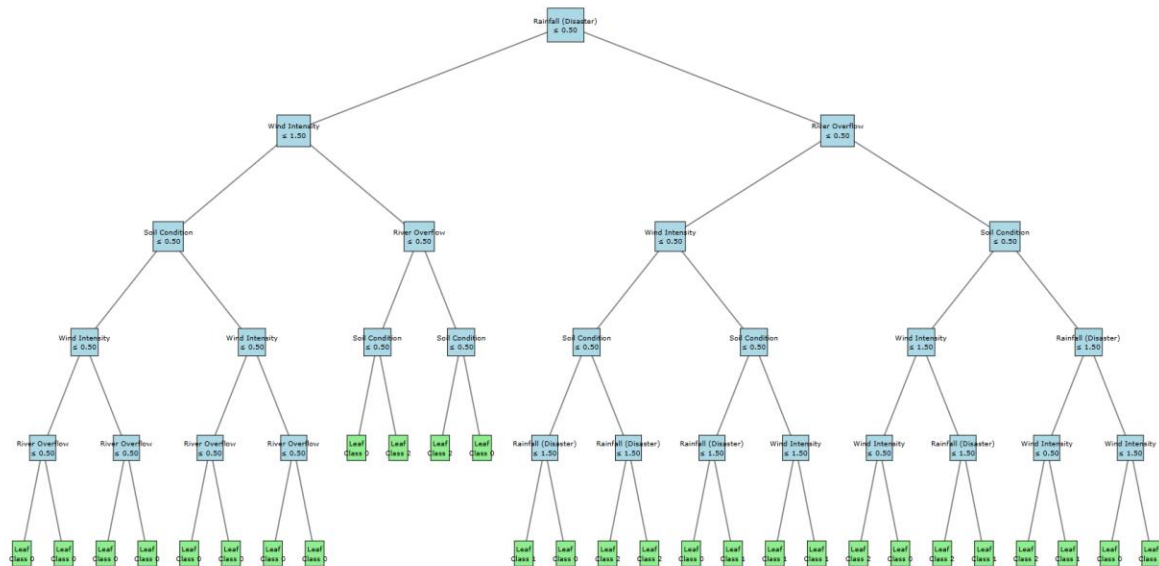
If Rainfall = Very Heavy → Severe Severity

The Decision Tree model used in this study classifies disaster severity based on four environmental predictors: **rainfall, river overflow, soil condition, and wind intensity**. Among these variables, the model determines which factor most strongly influences disaster outcomes through **Information Gain**, a measure of how well a feature separates the dataset into distinct classes.

After computing entropy and information gain, the results show that **Rainfall (IG = 0.745)** is the most influential feature. This indicates that rainfall levels provide the highest reduction in uncertainty when predicting disaster severity. As a result, Rainfall becomes the **root node** of the decision tree, meaning all classification decisions begin from this variable.

The remaining features—River Overflow, Soil Condition, and Wind Intensity—serve as secondary decision nodes that refine the prediction after rainfall is evaluated. This hierarchical structure reflects how environmental conditions interact in real-world disaster situations, where rainfall often triggers or intensifies other hazards such as flooding and soil weakening.

Decision Tree with 1000 datasets:



The resulting graph presents an interactive decision tree that visually explains how the model predicts disaster severity based on environmental factors such as rainfall, river overflow, soil condition, and wind intensity. Each box in the graph represents either a decision node or a final outcome, where decision nodes show the condition being evaluated (e.g., a feature and its threshold), and leaf nodes represent the predicted disaster severity class.

The connecting lines illustrate the flow of decisions from the root node down to the final predictions, showing how different combinations of environmental conditions lead to specific outcomes. The layout is hierarchical, allowing users to clearly trace each decision path step-by-step.

Additionally, when hovering over each node, detailed information such as sample size and class distribution is displayed, providing deeper insight into how the model arrived at each decision. Overall, the graph serves as a clear and interactive visualization of the decision-making process behind disaster severity prediction.

Implementation and Experimentation

The disaster dataset, composed of encoded survey responses, was processed and tested using **Naïve Bayes**, **K-Nearest Neighbors (KNN)**, and **Decision Tree** algorithms. This section outlines the practical steps taken during implementation and the evaluation of model performance.

1. Dataset Preparation

The dataset contained multiple disaster-related features such as rainfall intensity, water level, wind strength, soil condition, cloud density, humidity, and early warning system reception. Each categorical variable was systematically converted into numerical values (e.g., Heavy Rainfall = 4, Knee-level Water = 2, Strong Wind = 3). This encoding ensured compatibility with machine learning algorithms and enabled consistent computational analysis.

```
import pandas as pd

# Load dataset

data = pd.read_csv("disaster_dataset.csv")

# Features and labels

X = data.drop("DisasterLabel", axis=1)

y = data["DisasterLabel"]
```

2. Preprocessing

- **Missing values handling:** Incomplete or inconsistent survey entries were handled by either imputing missing values using the **median** or removing records with invalid/inconsistent responses (depending on the nature of the missingness).
- **Feature scaling: Min–Max normalization** was applied **only for KNN** to rescale the continuous/numeric inputs between 0 and 1. This ensured that distance calculations were not dominated by features with larger numeric ranges.
- **Train–test split:** The dataset was divided into **80% training data** and **20% testing data**. All models were trained using the same training set and evaluated on the same test set to allow fair comparison of their predictive performance.

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import MinMaxScaler

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Scaling for KNN

scaler = MinMaxScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

3. Model Training

- **Naïve Bayes:** Implemented as a probabilistic classifier using **Bayes' Theorem**. Prior probabilities and likelihoods were computed from the training set to predict **Typhoon Warning (Yes/No)**. Since Naïve Bayes is not distance-based, feature scaling was not required.
- **KNN:** Implemented using **Euclidean distance** as the similarity measure. The model tested multiple values of k (k = 3, 5, 7) to analyze how neighborhood size affects prediction of **Flood Risk (Low/Medium/High)**.
- **Decision Tree:** Constructed using **Information Gain (Entropy)** as the splitting criterion. The tree classified **Disaster Severity (Low/Moderate/Severe)** using the selected features: rainfall, river overflow, soil condition, and wind intensity. Scaling was not applied because the Decision Tree method relies on rule-based splits rather than distance metrics.

```
from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier

# Naïve Bayes

nb_model = GaussianNB().fit(X_train, y_train)

# KNN with different k values

knn3 = KNeighborsClassifier(n_neighbors=3).fit(X_train_scaled, y_train)

knn5 = KNeighborsClassifier(n_neighbors=5).fit(X_train_scaled, y_train)

knn7 = KNeighborsClassifier(n_neighbors=7).fit(X_train_scaled, y_train)

# Decision Tree

dt_model = DecisionTreeClassifier(criterion="entropy",
random_state=42).fit(X_train, y_train)
```

4. Prediction

Each algorithm was applied to the test set to generate predictions, and the predicted labels were compared against the actual labels:

- **Naïve Bayes** predicted **Typhoon Warning = Yes/No**
- **KNN** predicted **Flood Risk = Low/Medium/High**
- **Decision Tree** predicted **Disaster Severity = Low/Moderate/Severe**

```
# Predictions

nb_pred = nb_model.predict(X_test)

knn3_pred = knn3.predict(X_test_scaled)

knn5_pred = knn5.predict(X_test_scaled)

knn7_pred = knn7.predict(X_test_scaled)

dt_pred = dt_model.predict(X_test)
```

5. Evaluation

Model performance was measured using accuracy on the test set (same 20% split for all models):

- **Naïve Bayes** → Achieved **59.0% accuracy** in predicting *Typhoon Warning*.
- **KNN** → Accuracy varied depending on the value of k :
 - $k = 3 \rightarrow 49.0\%$
 - $k = 5 \rightarrow 49.0\%$
- $k = 7 \rightarrow 52.33\%$
 - The best performance was observed at $k = 7$, which was selected as the optimal KNN setting.
- **Decision Tree** → Reached **45.0% accuracy** in predicting *Disaster Severity*.

```
from sklearn.metrics import accuracy_score, confusion_matrix

# Accuracy

print("Naïve Bayes Accuracy:", accuracy_score(y_test, nb_pred))

print("KNN (k=3) Accuracy:", accuracy_score(y_test, knn3_pred))
```

```

print("KNN (k=5) Accuracy:", accuracy_score(y_test, knn5_pred))

print("KNN (k=7) Accuracy:", accuracy_score(y_test, knn7_pred))

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))

# Confusion Matrices

print("Naïve Bayes Confusion Matrix:\n", confusion_matrix(y_test,
nb_pred))

print("KNN (k=3) Confusion Matrix:\n", confusion_matrix(y_test,
knn3_pred))

print("KNN (k=5) Confusion Matrix:\n", confusion_matrix(y_test,
knn5_pred))

print("KNN (k=7) Confusion Matrix:\n", confusion_matrix(y_test,
knn7_pred))

print("Decision Tree Confusion Matrix:\n", confusion_matrix(y_test,
dt_pred))

```

6. Visualization

To support analysis and interpretation, results were summarized using:

- **Bar charts/tables** comparing the accuracy of the three algorithms.
- **Confusion matrices** (where generated) to show misclassifications and error patterns for each model, providing deeper insight into model behavior beyond overall accuracy.

Visualization and Analysis

This section presents the visualization and analysis of the experimental results. Graphs and tables are used to illustrate model performance in terms of accuracy, confusion matrices, feature importance, and evaluation metrics such as precision, recall, and F1-score. These visualizations provide deeper insight into how each algorithm performed on the grouped dataset and highlight the strengths and weaknesses of the models.

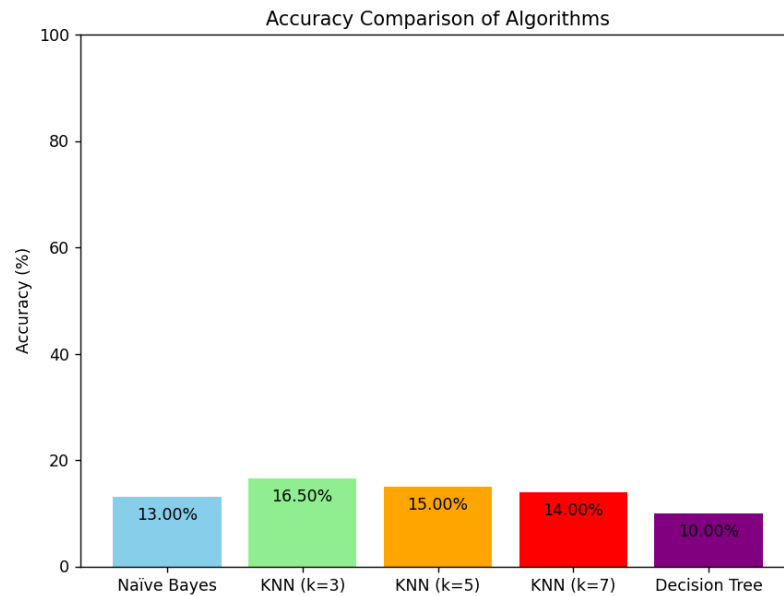
Preprocessing Comparison

Feature	Before Cleaning (Categorical)	After Cleaning (Numeric Encoding)
Cloud Density	Low, Medium, High	1, 2, 3
Humidity	Low, Medium, High	1, 2, 3
Rainfall (Typhoon)	Light, Moderate, Heavy	1, 2, 3
Wind Speed	Weak, Moderate, Strong	1, 2, 3
Typhoon Warning	No, Yes	0, 1
Rainfall Intensity	Very Light, Light, Moderate, Heavy, Very	1, 2, 3, 4, 5
Water Level	Below ankle, Knee, Waist, Chest, Above	1, 2, 3, 4, 5
Wind Strength	Weak, Moderate, Strong, Very Strong	1, 2, 3, 4
Flood Risk	Low, Medium, High	1, 2, 3
Rainfall (Disaster)	Light, Moderate, Heavy	1, 2, 3
River Overflow	No, Yes	0, 1
Soil Condition	Weak, Stable	1, 2
Wind Intensity	Weak, Moderate, Strong	1, 2, 3
Disaster Severity	Low, Moderate, Severe	1, 2, 3
Evacuation	No, Yes	0, 1
Early Warning	Never, Rarely, Sometimes, Always	1, 2, 3, 4
Disaster Label	Flood, Typhoon, Landslide, Earthquake, Storm Surge, Fire, Drought	Flood = 0
		Typhoon = 1,
		Landslide = 2,
		Earthquake = 3,
		Storm Surge = 4,
		Fire = 5,
		Drought = 6
Tech & ML Belief	Strongly Disagree ... Strongly Agree	1 ... 5
Grouped Label	Hydrological (Flood, Storm Surge),	1
	Meteorological (Typhoon, Wind-related)	2
	Geological (Earthquake, Landslide, Fire,	3

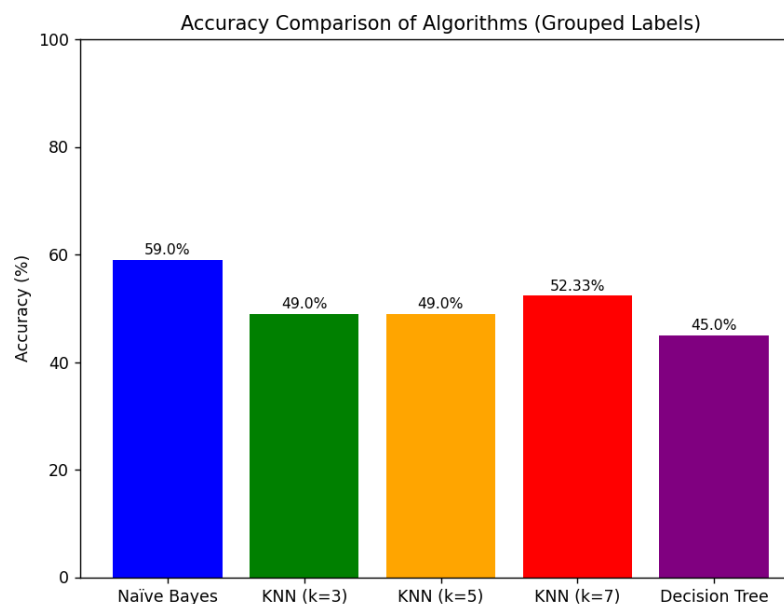
In the preprocessing stage, categorical disaster-related attributes were transformed into numeric encodings to make the dataset suitable for machine learning models. For example, rainfall intensity values such as *Light*, *Moderate*, *Heavy* were encoded as 1, 2, 3. This conversion ensures that algorithms like KNN, Naïve Bayes, and Decision Tree can interpret the features mathematically. Additionally, a **Grouped Label** column was introduced to consolidate the seven disaster types (Flood, Typhoon, Landslide, Earthquake, Storm Surge, Fire, Drought) into three broader categories: Hydrological, Meteorological, and Geological. This grouping improved model accuracy by reducing class imbalance and simplifying classification.

Accuracy Comparison

Without Grouped Label



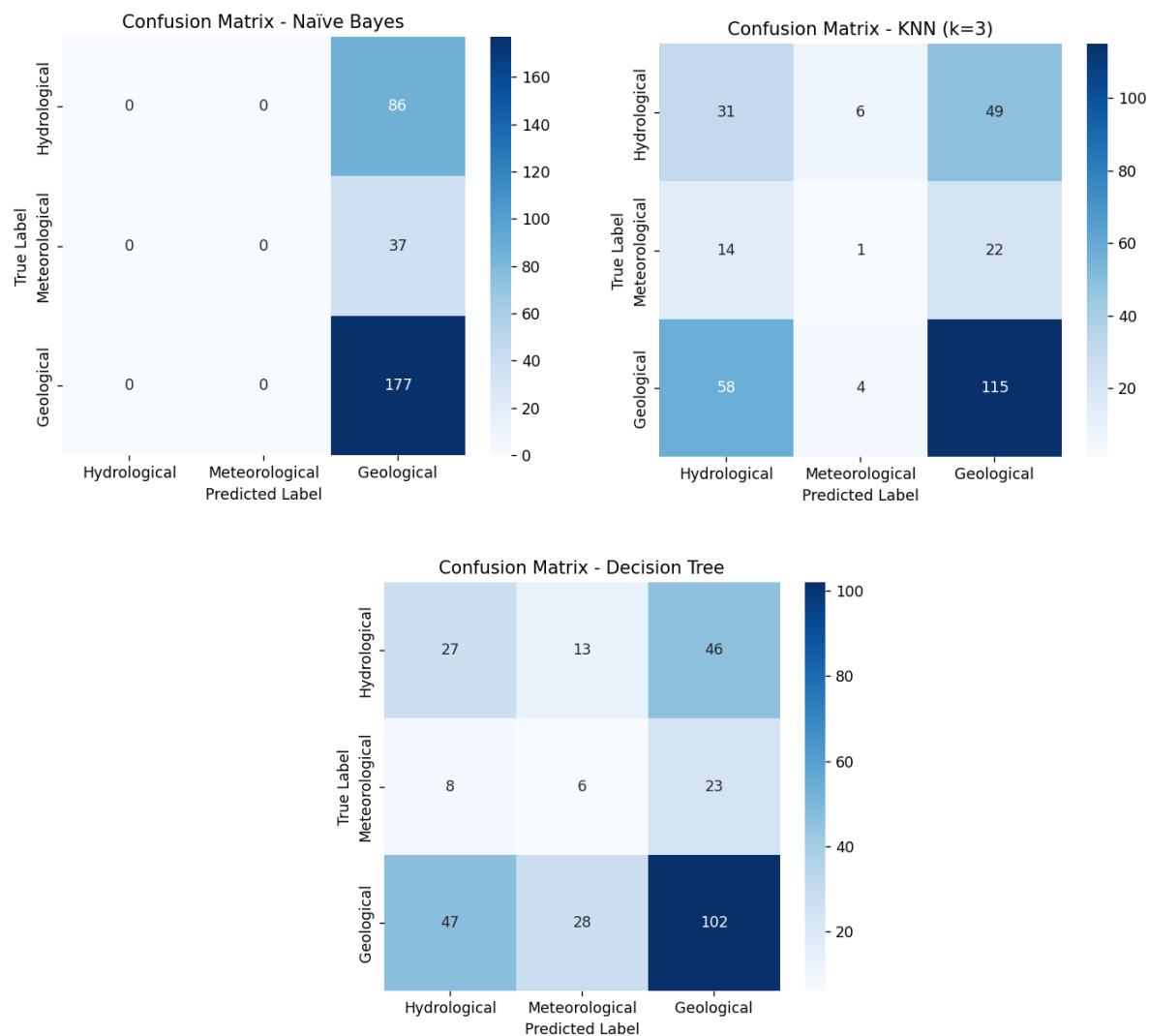
With Grouped Label



The accuracy comparison illustrates the performance of different machine learning algorithms in classifying disasters into Hydrological, Meteorological, and Geological

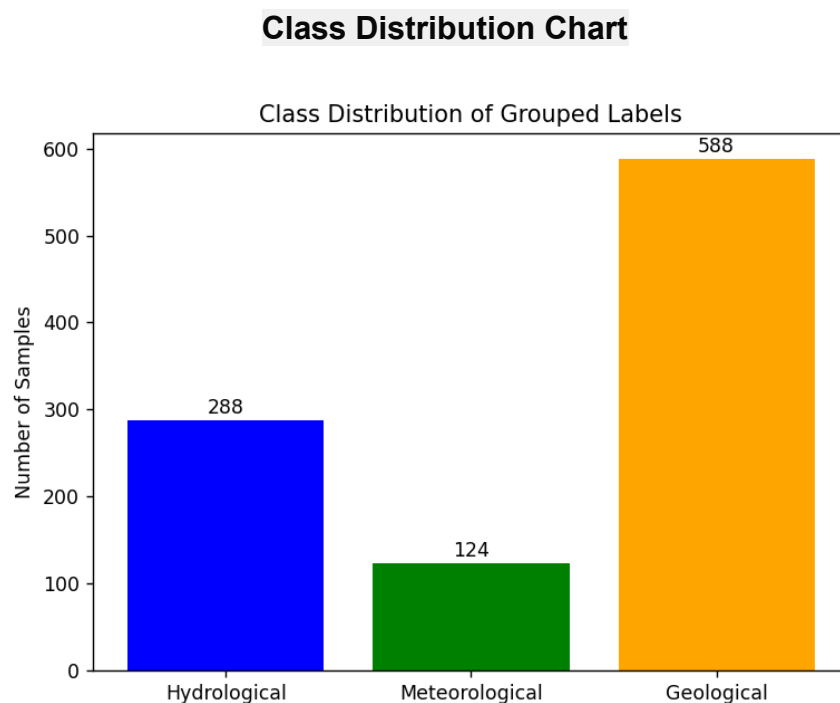
groups. Among the tested models, **Naïve Bayes achieved the highest accuracy (59.00%)**, while KNN variants ranged between **49.00%–52.33%**, and **Decision Tree reached 45.00%**. These results confirm that grouping labels improved predictive performance compared to the original seven-class dataset, where accuracy was much lower. Although the models did not reach very high accuracy, they consistently performed above the random baseline of 33%, showing that the dataset contains meaningful patterns for disaster classification.

Confusion Matrix



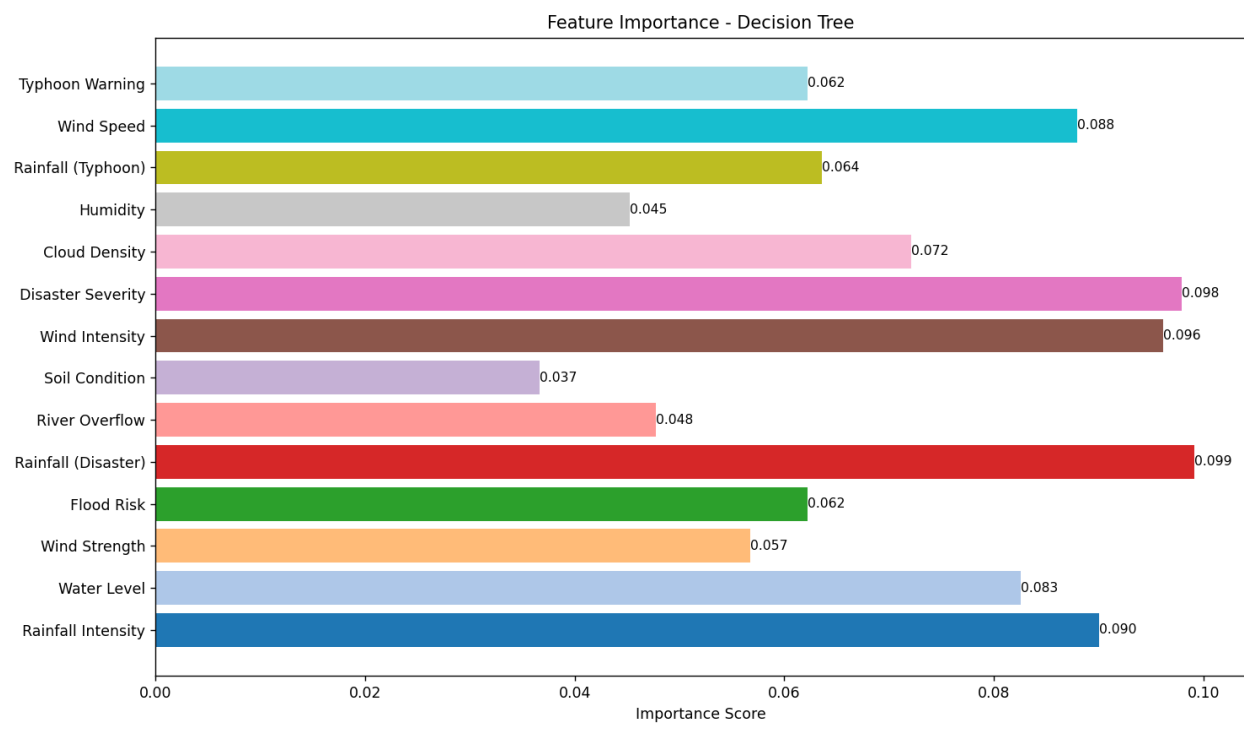
The confusion matrices highlight how each algorithm classifies disasters into Hydrological, Meteorological, and Geological groups:

- **Naïve Bayes** → Predicted almost all samples as *Geological*, showing strong bias and poor separation of categories. This explains why its confusion matrix is heavily skewed.
- **KNN (k=3)** → Distributed predictions more evenly, correctly identifying some Hydrological and Meteorological cases, but still misclassifying many into Geological. This shows KNN can capture local patterns but struggles with overlapping features.
- **Decision Tree** → Achieved a more balanced classification, correctly predicting across all three groups, though misclassifications remain. Its structure allows it to split based on feature thresholds, giving clearer decision boundaries compared to Naïve Bayes.



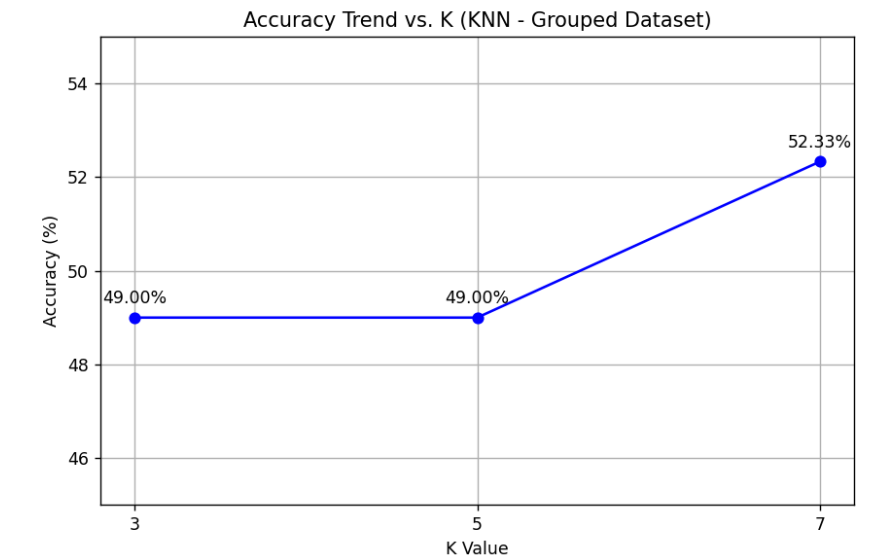
The class distribution chart shows that the dataset is **imbalanced**, with **Geological disasters (588 samples)** being the most frequent, followed by **Hydrological (288 samples)** and **Meteorological (124 samples)**. This imbalance explains why some models, such as Naïve Bayes, tend to predict Geological disasters more often. By grouping the original seven disaster types into three broader categories, the dataset becomes easier to classify, but the imbalance still affects accuracy. This visualization highlights the importance of preprocessing and grouping to improve model performance and interpretability.

Feature Importance (Decision Tree)



The feature importance chart shows that **Rainfall (Disaster)**, **Disaster Severity**, **Wind Intensity**, and **Rainfall Intensity** are the most influential variables in the Decision Tree model, each with relatively higher importance scores. These features highlight how rainfall patterns and wind conditions play a central role in distinguishing disaster categories. Moderate importance is also observed in **Water Level** and **Wind Speed**, which further support hydrological and meteorological classification. On the other hand, features such as **Soil Condition** and **Humidity** have lower scores, indicating they contribute less to the model's decisions. Overall, the chart emphasizes that rainfall and wind-related attributes are the dominant drivers of disaster classification in the Decision Tree.

Accuracy Trend vs K (for KNN)



The accuracy trend vs. k shows that KNN performance remained stable at **49%** for k=3 and k=5, then improved slightly to **52.33% at k=7**. This indicates that a larger neighborhood size helped the model generalize better, reducing noise from individual points. The upward trend suggests that **k=7 is the optimal setting** for this dataset, though the improvement is modest.

Evaluation Metrics: Precision, Recall, F1-score

Model	Class	Precision	Recall	F1-Score	Support
Naïve Bayes	1 (Hydrological)	0	0	0	58
	2 (Meteorological)	0	0	0	25
	3 (Geological)	0.585	1	0.738	117
	Macro Avg	0.195	0.333	0.246	200
	Weighted Avg	0.342	0.585	0.432	200
KNN (k=7)	1 (Hydrological)	0.364	0.345	0.354	58
	2 (Meteorological)	0	0	0	25
	3 (Geological)	0.587	0.692	0.635	117
	Macro Avg	0.317	0.346	0.33	200
	Weighted Avg	0.449	0.505	0.474	200
Decision Tree	1 (Hydrological)	0.27	0.293	0.281	58
	2 (Meteorological)	0.147	0.2	0.169	25
	3 (Geological)	0.602	0.53	0.564	117
	Macro Avg	0.34	0.341	0.338	200
	Weighted Avg	0.449	0.42	0.432	200

- **Naïve Bayes** → Completely failed to classify Hydrological and Meteorological disasters (precision/recall = 0), but performed very well on Geological (F1 = 0.738). Its weighted average F1 = **0.432** reflects this imbalance.
- **KNN (k=7)** → Improved recall for Geological (0.692) and gave moderate performance for Hydrological (F1 = 0.354), but still failed on Meteorological (0.000). Weighted F1 = **0.474**, slightly better than Naïve Bayes.
- **Decision Tree** → More balanced across classes, with moderate scores for Hydrological (0.281) and Meteorological (0.169), and decent performance on Geological (0.564). Weighted F1 = **0.432**, similar to Naïve Bayes but with better distribution across classes.

Overall, the results show clear differences in how the three models handle disaster classification. **Naïve Bayes** performed strongly only for Geological disasters but completely failed to classify Hydrological and Meteorological events. This is because it assumes feature independence, struggles with overlapping classes, and is highly affected by class imbalance, causing it to favor the most easily separable class while ignoring others.

K-Nearest Neighbors (KNN) showed slight improvement, especially in capturing local patterns, which helped its performance for Geological and partly for Hydrological

disasters. However, it still failed to identify Meteorological cases due to scattered and overlapping data distribution, making it inconsistent in separating all classes.

The **Decision Tree** provided the most interpretable results and was the only model that produced predictions for all disaster types. Although its accuracy is not the highest, it offers clearer decision rules and insights into feature importance.

Overall, Geological disasters were the easiest to classify across all models, while Hydrological and Meteorological disasters were more difficult due to class imbalance and overlapping patterns. These findings suggest the need for better feature selection and techniques to address data imbalance to improve overall model performance.

Appendices

Source Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import MinMaxScaler

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier

import tkinter as tk

from tkinter import simpledialog, messagebox

# -----

# 1. Load Dataset

# -----

#File save to a folder (disaster) in desktop

data = pd.read_excel("C:/Users/ASUS/OneDrive/Desktop/disaster/grouped_dataset.xlsx")

# Drop both label columns (targets)

X = data.drop(["Disaster Label", "Grouped Label"], axis=1)

y = data["Disaster Label"] # Use Disaster Label as target
```

```

# -----

# 2. Preprocessing

# -----

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# -----

# 3. Train Models

# -----

nb_model = GaussianNB().fit(X_train, y_train)

knn_model = KNeighborsClassifier(n_neighbors=3).fit(X_train_scaled, y_train)

dt_model = DecisionTreeClassifier(criterion="entropy",
random_state=42).fit(X_train, y_train)

# -----

# 4. GUI Input (Survey Questions)

# -----

root = tk.Tk()

root.withdraw()

messagebox.showinfo("Disaster Prediction", "Please answer the following survey
questions (type the numeric code).")

# KNN Features

rainfall_intensity = int(simpledialog.askstring("Input", "During heavy
rainfall, how would you describe the rainfall intensity in your area? (1=Very
Light, 2=Light, 3=Moderate, 4=Heavy, 5=Very Heavy)"))

```

```

water_level = int(simpledialog.askstring("Input", "How high does flood water
usually rise in your area? (1=Below ankle, 2=Knee, 3=Waist, 4=Chest, 5=Above
chest)"))

wind_strength = int(simpledialog.askstring("Input", "During storms or typhoons,
how strong is the wind usually experienced? (1=Weak, 2=Moderate, 3=Strong,
4=Very Strong)"))

flood_risk = int(simpledialog.askstring("Input", "Based on your experience,
what is the flood risk level in your area? (1=Low, 2=Medium, 3=High)"))

# Decision Tree Features

rainfall_disaster = int(simpledialog.askstring("Input", "How would you classify
rainfall during disaster events in your area? (1=Light, 2=Moderate, 3=Heavy)"))

river_overflow = int(simpledialog.askstring("Input", "Does river overflow
usually occur during heavy rain? (0=No, 1=Yes)"))

soil_condition = int(simpledialog.askstring("Input", "What is the condition of
the soil in your area during heavy rain? (1=Weak, 2=Stable)"))

wind_intensity = int(simpledialog.askstring("Input", "How would you describe
wind intensity during disasters? (1=Weak, 2=Moderate, 3=Strong)"))

disaster_severity = int(simpledialog.askstring("Input", "Based on your
observation, how severe are disasters in your area? (1=Low, 2=Moderate,
3=Severe)"))

# Naïve Bayes Features

cloud_density = int(simpledialog.askstring("Input", "Before a typhoon occurs,
how would you describe cloud density? (1=Low, 2=Medium, 3=High)"))

humidity = int(simpledialog.askstring("Input", "How would you describe humidity
before heavy rain or typhoon? (1=Low, 2=Medium, 3=High)"))

rainfall_typhoon = int(simpledialog.askstring("Input", "How would you classify
rainfall during typhoon events? (1=Light, 2=Moderate, 3=Heavy)"))

wind_speed = int(simpledialog.askstring("Input", "How would you classify wind
speed during typhoons? (1=Weak, 2=Moderate, 3=Strong)"))

typhoon_warning = int(simpledialog.askstring("Input", "Based on weather
conditions, do you think a typhoon warning should be issued? (0=No, 1=Yes)"))

# -----

# 5. Predictions with Full Probabilities

# -----

```

```

user_input = [[rainfall_intensity, water_level, wind_strength, flood_risk,
               rainfall_disaster, river_overflow, soil_condition,
               wind_intensity, disaster_severity,
               cloud_density, humidity, rainfall_typhoon, wind_speed,
               typhoon_warning]]

# Scale for KNN

user_input_scaled = scaler.transform(user_input)

# Probabilities for each class

nb_probs = nb_model.predict_proba(user_input)[0]

knn_probs = knn_model.predict_proba(user_input_scaled)[0]

dt_probs = dt_model.predict_proba(user_input)[0]

# -----

# 6. Map numeric codes to disaster names

# -----

disaster_map = {
    0: "Flood",
    1: "Typhoon",
    2: "Landslide",
    3: "Earthquake",
    4: "Storm Surge",
    5: "Fire",
    6: "Drought"
}

# -----

# 7. Format Results with Percentages

# -----

def format_probs(probs, model_name):

```

```

lines = [f"{model_name} Probabilities:"]

for i, p in enumerate(probs):

    lines.append(f" {disaster_map[i]} → {p*100:.2f}%")

# Highlight the most confident prediction

best_idx = probs.argmax()

lines.append(f"Most Confident Prediction → {disaster_map[best_idx]}
({probs[best_idx]*100:.2f}%)")

return "\n".join(lines)

result_message = f"""

=== Prediction Results (Full Breakdown) ===

{format_probs(nb_probs, "Naïve Bayes")}

{format_probs(knn_probs, "KNN (k=3)")}

{format_probs(dt_probs, "Decision Tree")}

"""

# Show the full breakdown first

messagebox.showinfo("Prediction Results", result_message)

# -----

# 8. Final Consensus (Most Confident Overall)

# -----

# Collect all top predictions with their confidence

all_predictions = [

    ("Naïve Bayes", disaster_map[nb_probs.argmax()], nb_probs.max()),

    ("KNN", disaster_map[knn_probs.argmax()], knn_probs.max()),

    ("Decision Tree", disaster_map[dt_probs.argmax()], dt_probs.max())

]

# Find the single most confident prediction across all models

```



```
final_model, final_label, final_conf = max(all_predictions, key=lambda x: x[2])

# Show final popup

messagebox.showinfo("Final Prediction", f"{final_label} ({final_conf*100:.2f}% confidence from {final_model})")
```

Feature Mapping

KNN

Rainfall Intensity → During heavy rainfall, how would you describe the rainfall intensity in your area?

Water Level → How high does flood water usually rise in your area?

Wind Strength → During storms or typhoons, how strong is the wind usually experienced?

Flood Risk → Based on your experience, what is the flood risk level in your area?

Decision Tree

Rainfall → How would you classify rainfall during disaster events in your area?

River Overflow → Does river overflow usually occur during heavy rain?

Soil Condition → What is the condition of the soil in your area during heavy rain?

Wind Intensity → How would you describe wind intensity during disasters?

Disaster Severity → Based on your observation, how severe are disasters in your area?

Naïve Bayes

Cloud Density → Before a typhoon occurs, how would you describe cloud density?

Humidity → How would you describe humidity before heavy rain or typhoon?

Rainfall → How would you classify rainfall during typhoon events?

Wind Speed → How would you classify wind speed during typhoons?

Typhoon Warning → Based on weather conditions, do you think a typhoon warning should be issued?

References

- Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
<https://doi.org/10.1016/C2009-0-61819-5>
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
<https://www.cs.cmu.edu/~tom/mlbook.html>
- Zhang, H. (2004). The optimality of Naïve Bayes. *Proceedings of the Seventeenth International Florida Artificial Intelligence Research Society Conference*, 562–567.
<https://www.aaai.org/Papers/FLAIRS/2004/Flairs04-097.pdf>
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
<https://doi.org/10.1109/TIT.1967.1053964>
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
<https://doi.org/10.1007/BF00116251>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<http://jmlr.org/papers/v12/pedregosa11a.html>